

Coding and Configuration for Secure Systems in BSDs

John O'Keefe-Odom

Contents

Author	iii
Preface	v
Acknowledgements	vii
Introduction	ix
1 Amnesiac Systems in FreeBSD	1
1.1 Introduction	1
1.1.1 Background and Content	1
1.1.2 Problem Statement	1
1.1.3 Project Objectives	1
1.1.4 Scope of Work	1
1.1.5 Variables	1
1.1.6 Assumptions, Risks, and Limitations	1
1.1.7 Significance of the Project	1
1.2 Literature Review	1
1.2.1 Overview of Existing Solutions or Approaches	1
1.2.2 Comparisons of Technologies or Frameworks	2
1.2.3 Gaps in the Literature	2
1.3 Methodology	2
1.3.1 Technical Research Questions	2
1.3.2 System Architecture and Design	2
1.3.3 Implementation Plan	2
1.4 Analysis	21
1.4.1 Quality	21
1.4.2 Tests	21
1.5 Discussion	21
1.6 Chapter References	21
1.6.1 Books	21
1.6.2 MAN Pages	21
1.6.3 Websites	22
Endnotes	23
2 FreeBSD Tape Operations	29
2.1 Introduction	29
2.2 Methodology	29
2.3 Results	29
2.3.1 Code Notes	29
2.4 Analysis	33
2.4.1 Quality	33

2.4.2	Tests	33
2.5	Discussion	33
2.6	References	34
Endnotes		35
Glossary		37
Endnotes		41
Index		43

Author

John O’Keefe-Odom received a Master of Liberal Arts (ALM) in Extension Studies in the field of Cybersecurity from Harvard University, Harvard University Extension School, in 2026. He received an AAS in Web Programming from Chattanooga State Community College in 2013. He received a BA in English: Writing from the University of Tennessee at Chattanooga in 2001. He holds various technical certifications in cybersecurity including: Certified Information Systems Security Professional. (CISSP), Certified Secure Software Lifecycle Professional. (CSSLP), Certified Cloud Security Professional. (CCSP), SecurityX, PenTest+, CySA+, Security+, and Linux+. He has worked as a senior application security engineer and software engineer for various companies for over ten years.

Preface

Your preface text goes here...

Acknowledgements

Your acknowledgements text goes here...

With our chosen style of citations, we tried to provide references that would withstand common lab use and academic rigor. We provided in-chapter sections for convenient references to books, man pages, and websites. We also continued with inline footnoting for chapter endnotes; we especially noted various computer commands and technologies since they are fundamentally derivative works upon which we depend. This made for a lot of references and some duplicity. We have cited generously.

We decided to set a previous book in fonts by Erik Spiekermann after seeing “Helvetica” by Gary Hustwit (2007). We carried that through here. InfoOTDisp-Book is copyrighted by Ole Schaefer and Erik Spiekermann; it is published by FSI Fonts und Software GmbH. Eurostile Extended Regular and Eurostile W01 Regular are copyrighted 2006 and 2010 by URW Design and Development.

Introduction

Your introduction text goes here...

1 Amnesiac Systems in FreeBSD

In this technical note, we explore indexconfigurationconfigurations of the FreeBSD operating system. With a goal of creating an glsamensiac system that keeps the operating system separate from working storage drives, we experiment with various indexconfigurationconfigurations. We demonstrate the planning of drive indexconfigurationconfiguration indexcodingcoding and the reasoning behind the choices. First, we code a system that uses the operating system on the storage drive discs. Then we evolve the plan into using a live cd for operating system use with drives that are connected to the glsimmutable system in Random Access Memory. (RAM) via configuration code. Using an amensiac system like this, we would be able to reset the operating system to a known immutable state with every startup. The drives could be disconnected and reattached to other systems for various purposes. Throughout, we cover specific commands for drive and operating system management and operation.

1.1 Introduction

1.1.1 Background and Content

1.1.2 Problem Statement

1.1.3 Project Objectives

1.1.4 Scope of Work

1.1.5 Variables

1.1.6 Assumptions, Risks, and Limitations

1.1.7 Significance of the Project

1.2 Literature Review

1.2.1 Overview of Existing Solutions or Approaches

We came across an article ("Immutability as an Operational Constraint Rather Than a Security Feature") that pointed out that immutability can be a limiting factor. The author warned, "By just making something immutable you haven't necessarily made it safe; what you have effectively done is taken away the possibility of alteration and replaced it with a new operational reality: correction must happen through additional writes, and recovery must happen through reconciliation rather than repair"¹

In this approach, the entire system was assumed to be an immutable system. This helped us to realize what a benefit it was to not make the entire system immutable. Our original intention was to have centralized, core, aspects of the operating system be immutable; other parts of the system could be changed within the

rules and limits set by the OS.

Still, those changeable parts might have logging and monitoring activities, such as database snapshots set up in a way that allowed these cautions about immutability to contribute to our planning. Mallikarjun Vppalapati pointed out problems that could be had when records of on-system actions became immutable. Suppose a log or record was contaminated with malware or privacy information that needed to be forgotten under GDPR. Administrators might need to undertake complicated edits to allow future auditors to bypass data that should not be considered. Vppalapati outlined a situation in which a simple programmatic error needed an elaborate and complicated fix because simple overwrites were not available.²

Vppalapati described three kinds of immutability: hard, soft, and operational. "Hard immutability describes a state of being unchangeable where changes are so heavily protected by technical mechanisms and therefore practically infeasible the alteration process is therefore practically impossible."³ "Soft immutability...can be reverted by using privileged credentials."⁴ "Operational immutability implies the lack of changes without the use of technology"⁵ In our proposed system, we would use hard immutability in the optical read-only recording of the core OS. We would use operational immutability as a control because we were applying an optical disc that required physical access to the server for alterations. We would also use soft immutability, in the writing of data to files in attached storage that was not automatically loaded by the system on boot.

1.2.2 Comparisons of Technologies or Frameworks

1.2.3 Gaps in the Literature

1.3 Methodology

1.3.1 Technical Research Questions

1.3.2 System Architecture and Design

1.3.3 Implementation Plan

We're starting with a Dell T610 Server. Made in 2011, this machine is about ten years old. We purchased this at a government e-waste auction. She needed some cleanup. We started on the front lawn, blowing out the dust with a leafblower.

Most of the time servers from these auctions will have minimal RAM. Usually the drives and any good accessories will be stripped out. Anytime we get a hard drive, from any source, one of the first actions we take is to erase the drives to National Institute of Standards and Technology. (NIST) 800-88 Purge⁶ with dedicated devices. That kind of erasing can be done with `dd` in Kali or FreeBSD; but, the drive erasers usually have a verification feature that's helpful.

Hardware Configuration

This is my favorite computer. I've named her "Blondie" after Debbie Harry's New Wave band.⁷

Now, she's configured with:

- A DVD RW drive
- A LTO III⁸ tape backup unit
- In her 8 drive bay, we have:
 - Two VelociRaptor⁹ 80GB 10000 rpm drives.
 - ★ We'll use these in a Mirror configuration to hold the main OS.
 - Two refurb 120GB Solid State Drive. (SSD)s
 - ★ We'll use this in a Mirror as ZFS log drives.
 - ★ They'll help to make writes faster.
 - Four 500GB hard drives
 - ★ We'll use this as a RAIDZ-2 array to hold the jails.
 - ★ We have some smaller Serial Attached SCSI. (SAS) drives.
 - ★ We have some Serial AT Attachment. (SATA) drives.
- We replaced the one CPU with two Xeon 5550¹⁰ quad cores and added a cooling tower.
- We added more RAM. Currently, we're at 196GB.
- We added some more Network Interface Card. (NIC)s.
 - We have four for the jails.
 - We have two for the OS.
 - We'll leave the two on the motherboard mostly unused, except for our iDRAC connection.
 - ★ iDRAC is a proprietary baseboard controller.
 - ★ We could use it to power cycle the machine.
 - ★ There's a web interface that let's us know about motherboard-related facts.
 - ★ We cannot give a GELI password through the console in our current setup.
- The server has two 500W power supplies.

Because the dual power supplies draw about 500 Watts apiece, this server gets its own circuit.

The drives we're using are older, but we got enough copies of the drives to be able to support backups. When making physical backups of the drives, it can be helpful to have other volumes of the same model. So, instead of getting the best available, we planned for the best we could afford at quantity.

The server does not have:

- Any type of graphics card or Graphics Processing Unit. (GPU)
- Sound cards or microphone hardware
- Wireless networking or Bluetooth.

Intentions: Paving the Road to Jails

We'll use the box for development. It'll be our main development server. We'll use it with progRAMs that we compile on a nearby *poudriere* package building box. In those package builds, we have segregated our builds into a pattern of jails that mimic the types of jails that will be needed on most of our computers.

So, there is a jail for functional groups like: development tools, offensive and defensive security programs, browsers, desktop software, and so on. *Poudriere* jails can be filled with whatever sets of programs we like. For successful builds, we find its best to keep the jails small.

There will be three main groups of drives:

- an OS Mirror pair
- a ZFS log Mirror pair
- a ZFS RAIDZ-2 kit of four drives.

The OS pair will hold ideas like:

- jail configuration
- main host networking and Virtual Network. (VNET)
- host-based user accounts and configuration.

The ZFS kits will hold disconnected jails. We'll set up a system of hierarchical jails that provide us with a development environment made up of modular jails that will be the children of an overarching parent. These jails will communicate with each other and the parent through some custom VNET. We'll do all of our jail scripting, controls, and configuration manually. Likewise, the base ZFS setup will be done with simple scripts.

Preparing the Baseboard

The T610 has a built-in RAID card that can't be totally bypassed. This means that since we'll want ZFS to control the drives as much as possible, we'll need to tell the hardware RAID to buzz off. We won't want hardware RAID to participate in the process. We'll just use the software RAID through ZFS.

We can't just unplug the RAID card. We tried that, and it leads to malfunctions on this machine. So, as a compromise, what we'll tell the machine is that each drive is its own VDEV and it's RAID-0. We'll end up with as many VDEVs as we have physical drives.

To prepare for a later step in this process, we'll note the serial number of each drive we load into position, its model number, and its capacity. We'll need to know that information, and we won't want to be hassled by having to power cycle the machine in the middle of the setup process just to check on those details.

Later on, we'll set VDEVs with ZFS; but, from this point on, we'll basically ignore the RAID card's description of these devices as VDEVs.

With the baseboard prepared, and the BIOS set to allow us to boot from disc, we'll get started with a plain install of FreeBSD to the first two drives in a Mirror configuration.

Incremental Advancement to Amnesiac System

With all of the manual coding and configuration needed to set up an amnesiac system like the one we desired, it was a big change from the normal BSD installs we had done for several years. Our first pass at building an amnesiac system decayed into manually scripting a regular operating system installation that had most of the features we desired in the amnesiac system. Both the on-disc OS system and the amnesiac system are detailed below.

Identifying and Allocating ZFS Drive Qualities

We need to be able to detect the drives and partitions that we want to work with using `geom`, `gpart`, and `zfs` commands. If, for some reason, you can't find those drives, then you need to address that. While working with this particular computer, I've found that the leading cause of failing to detect a drive goes back to the `VDEV` assignment in the hardware RAID. If the drive were pulled, this particular RAID card will need to have those configuration changes addressed. Often, this means rebooting the machine. It could be done through the iDRAC baseboard controller in some models; but the one we have here is not configured to do those tasks.

What are we going to want the ZFS kit to do? How are we going to use the drives? How can we wrestle with this new type of configuration?

After reading over *FreeBSD Mastery: ZFS* by Alan Jude and Michael Lucas³⁸, we came to a few conclusions. We chose to use RAIDZ-2 with Log drives because of the number of drives we have³⁹. Some of the topics covered in that book that we'll want to apply include ideas in Table 1.1. In our first iterations we decided to install the OS on the hard drives in a pattern that looks like a conventional installation. Once the drive partitioning and utilization commands were worked out, we then started over and undertook the amnesiac version with the OS on a live disc that runs with attached drives.

Our goal for the system will be to split up the drives into three main groups, as described in Tables 1.2, 1.3, and 1.4. On each drive, we'll also allow:

- 15% of the disk's own capacity as "Free Space" by allocating a blank partition
- Swap by 100 | 50 | 33 | 25 percent of RAM (with a 25% minimum for all disks) or similar
- Remainder as one large ZFS main partition.

We decided we would want those qualities in an amnesiac system, with the exception of running the OS from a live disc.

For the OS-On-Disk system, this should bring us:

- Enough room for the main OS
- A 4 disk RAIDZ-2 array of almost 1.49 TB of actual storage
- Fast swap and copy-on-write
- With each disk capable of expansion or change.

The unused space on the drives (a blank partition of 15%) may look like we're grossly under-utilizing the disks. We're planning for future emergencies. Do we need more

Table 1.1: ZFS Concepts and Commands

Topic	Concept	Command phrase example
Disk labels	Use the label to associate the logical with the physical ¹¹	[Host]-[PositionNumber 0-7]-[Serial_Number] ¹²
ZFS Compression	Faster performance: less space ¹³	zfs set compress=gzip-6 [DIRECTORY] ¹⁴
ZFS zpool comment quota ¹⁷	Make a note on the pool ¹⁵ Limit growth ¹⁸	zfs set comment=""- ¹⁶ -
reservation ¹⁹	Set aside space for directory and its children ²⁰	-
refreservation ²¹	Set aside space but don't count snapshots and child datasets ²²	-
canmount ²³	Control mounting ²⁴	zfs set canmount=off [DIRECTORY] ²⁵
mountpoint ²⁶	Specify where to mount ²⁷	zfs set mountpoint=/SOME [DIRECTORY] ²⁸
readonly ²⁹	Control readonly ³⁰	zfs set readonly=off [DIRECTORY] ³¹
exec ³²	Control if it can execute ³³	zfs set exec=off [DIRECTORY] ³⁴
copies ³⁵	Make more than one copy ³⁶	zfs set copies=2 [DIRECTORY] ³⁷

Table 1.2: ZFS Main OS Use Hard Disk: Initial Estimates

	Estimate for 80 GB HDD
Blank: 15% "Free Space"	12 GB
Swap: Under-allocate, deliberately	10 GB
ZFS Main Partition: Remainder	58 GB

Table 1.3: ZFS jail Use Hard Disk: Initial Estimates

	Estimate for 500 GB HDD
Blank: 15% "Free Space"	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

swap? Maybe we can add it in. Do we need to expand or resize the disk? Maybe we can delete that blank "Free Space" partition and use that. By not trying to use everything, we're giving ourselves some room to solve future problems.

What We Can Learn From Installing an OS

Later on, we'll install the other drives with `gpart` and `zfs` commands. This first pair, through, we could do with `bsdinstall`⁴⁰. If we choose that, though, then we must commit the whole disk to the process. That would be how we would normally stand up a disk drive. In this case, we'll conduct a manual process that mimics the main steps of `bsdinstall`⁴¹ to show that the installation can be done manually. Part of these same kinds of steps are critical tasks when administering and installing jails.

Basic Operating system Installation Goals

We're going to do an early phase, basic installation of the OS. At this point, we'll really only set up a pair of users and some plain vanilla settings for the OS. We want to be able to:

- to establish our ZFS kit
- to configure our main jail relationships
- to be able to check our networking with other hosts

Table 1.4: ZFS SLOG Log Use SSD: Initial Estimates

	Estimate for 120 GB SSD
Blank: 15% "Free Space"	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

- to set the basic operating system to call packages from our package server.

We can come back later and make a more sophisticated environment. At this stage, we will just lay the foundation for that work. If we don't get the ZFS kit stood up, then our more complex work with jails won't have an environment in which to thrive.

Main Steps for FreeBSD Installation from `bsdinstall`

42

We can see that `bsdinstall`⁴³ is broken up into about a dozen main segments. They are:

- Keyboard mapping
- Hostname
- Networking configuration
- Wireless networking
- Partitioning
- Mounting temporary root
- Fetch the OS
- Extract the OS
- Set a root password
- Set the time
- Add users
- Enable some services
- Copy in the config [2].

We're not going to do all of these. We'll start with partitioning. We won't do any wireless networking or set a root password. We're going to skip the wireless networking for now because the hardware doesn't have that capability. In the newer versions of `bsdinstall`, there are some protective measures not listed above. We won't tackle those just yet.

No Root Password

We won't set a root password because we will expect to always Secure Shell. (SSH) into the root account with a key. By not setting a root password, we will create a situation where a key is always required. This can be dangerous in that it can create a single point of failure for the system.

Another risk is that we will require the ability to SSH in as root. Later, as we establish software firewalls on the host, we will want to limit what addresses can be used to access this root with SSH. It's a little dangerous, but it's also a little different.

We have some risk controls that will be in place. First, we often store our keys in a special collection of USB drives. Next, we will have 2FA on the SSH that will require a TOTP (Time-based One Time Password). Finally, we will limit what's done in this drive. Since most of our programs and work will be done in the jails, there is a limited amount of influence that this part of the system will have. It's there to start the box and spin up the jails. If we ever have to totally destroy this installation and hook up our jails to another system, that is possible.

What do we gain from no root password? An extra measure of security. If there

is no password, then password enumeration attempts should all fail. This not only goes for root, but it goes for other users. Many of the automated login attacks we've seen have been simple dictionary attacks; these brute force exercises rarely attempt to account for the types of login procedures that we expect to put in place.

Not having a root password may also truncate or stop our ability to use commands like `su` or `sudo`. As we're trying to stop privilege escalation, we'll also be stopping legitimate means of priv-esc. It could be inconvenient, but we've used this technique before.

Also, there is another risk: leaving the needed key lying around. For example, if we give the key to a user account operator, like ourselves, and then we import copies of the key to the box for easy use: well, then those keys could be used and re-used locally. This would be the equivalent for storing a plain-text password on the box. They keys are ultimately simple text files with decorated, long, strings of hashes. It's the software that goes with the keys, combined with the configuration of the programs for logging in to a service, that makes them special. Switching from passwords to keys is not a perfect answer; but, it's the answer we'll use this time.

If someone wishes to put a root password in place, then that's a procedure that will be easy to implement. For now, though, there will be no root password. And that means we have to build the keys we will use to log in.

Be Able to Build the Keys

We can build our keys using the live disc's shell. and a thumbdrive. The main command to build the keys is:

```
ssh-keygen -b 1024 -t rsa44
```

Copy the files output to a thumbdrive and keep them handy for mounting.

Partition, Mount, and Use a Thumbdrive from Live Disc's Shell

Let's take some time to go over some detailed instructions on how to build those keys and get them onto a thumbdrive. The live disc's shell mostly uses a read-only file system. So, some people may not be familiar with how we are going to get the keys made and saved anywhere that will persist.

Start the Box and the Live Disk's Shell

With the hardware booted to the LiveCD (installation disc), we select "Shell" from the three main choices in the first menu. This drops us into the shell. If we try to save a file of any kind, we'll probably get an error message back that says we're in a read-only file system. Our main set of directions for this is in the FreeBSD Handbook. We look carefully at the sections for USB storage and "Adding Disks"

Be Able to Plug in the USB Thumbdrive and Find Its Logical Address

```
camcontrol devlist45
```

This should show the USB drive. If the list is long, we can pipe it to more and slowly advance through the list. In our case, the USB was at da0.

The listing should also show any other storage device that's plugged in to the machine. So, all of those hard drives loaded into the bays should appear. If they don't, then take it as a warning sign that we're having a problem with the RAID card's physical drive VDEVs. On the T610, we were seeing drives in positions 0 to 7 in addition to the USB thumbdrive we're working on now.

Be Able to Partition the USB

Since our USB has not been prepared for use with anything, we have to partition it. We can pretty much use the default commands from 17.2 "Adding Disks" to set up the drive. Here, we create a simple UFS file system and mount it to a directory that we'll put in /tmp. If our USB is already partitioned, we would want to skip this part.

```
gpart create -s GPT da046  
gpart add -t freebsd-ufs -a 1M da047  
newfs -U /dev/da0p148
```

Mount the USB to the /tmp Directory:

```
mkdir /tmp/someDir49  
mount /dev/da0p1 /tmp/someDir50
```

Be Able to Write to the USB

We could now write a simple text file onto /tmp/someDir, unmount the drive, and then re-mount it to check our work. That would go like:

```
cd /tmp51  
umount /tmp/someDir52  
rm -r /tmp/someDir53
```

Unplug the drive. Plug it in another slot. Use camcontrol devlist to find it again. It'll probably get the same address, like "da0".

```
mkdir /tmp/otherDir54  
mount /dev/da0p1 /tmp/otherDir55  
ls /tmp/otherDir56
```

Be Able to Put Some Key Pairs on the USB

Using these same principles, we can mount our USB thumbdrive, generate SSH keys, and save them there for later.

```
mkdir /tmp/otherDir/keys57
ssh-keygen -b 1024 -t rsa58
```

During the ssh-keygen dialog, simply direct the program to save the keys to /tmp/otherDir/keys. Be sure to record your passphrase somewhere.

Be Able to Use Commands to Build Out the Drives

We'd like to show a quick overview of some of the drive-related commands. These will be applied soon.

To list what devices are detected by the OS, we can use:

```
gpart show59
```

We will want to note the serial number of each drive, by position. We then note which drives we want to use for the main body of the RAIDZ-2 kit. All of this will become one pool of ZFS VDEVs. Then we'll add in the log mirrors. So, in the sequence of 0-7, numbering the drives, we should expect to skip those two for now.

Make the notes if you haven't already because we're going to list all of the drives for the main pool in one pop. This task may have been done before we set up the physical VDEVs for the hardware RAID.

We're going to choose to put a swap partition on the SSDs. We can add one onto the other disks, but a maximum of 4 swap partitions is recommended in the FreeBSD Handbook. Later on, we will be able to select swap partitions by mounting them and using commands like `swapon` `swapoff`.

To put the ZFS partition on the remaining disks:

```
gpart create -s SCHEME DISK_DEVICE60
```

```
gpart create -s gpt da461
```

To add the main ZFS partition:

```
gpart add -a 1m -l ZFS_NUMBER \
-t freebsd-zfs DISK_DEVICE62
```

```
gpart add -a 1m -l ZFS-BLONDIE-4-WD123465789 \
-t freebsd-zfs da463
```

To add the partition's swap:

```
gpart add -a 1m -slg -l SWAP_NUMBER \
-t freebsd-swap DISK_DEVICE64
```

```
gpart add -a 1m -slg -l SWAP-BLONDIE-4-WD123465789 \
-t freebsd-swap da465
```

To add a logging partition on the SLOG SSDs:

```
gpart add -a 1m -l ZLOG_NUMBER \
-t freebsd-zfs DISK_DEVICE66
```

```
gpart add -a 1m -l ZLOG-BLONDIE-2-WD123465789 \
-t freebsd-zfs da267
```

With each disk partitioned, we can add them to the pool for our RAIDZ-2 kit.

```
zpool create RAID POOL_NAME DISK_LABEL/ZFS_PARTITION_LABEL
...next disk/partition...68
```

```
zpool create BLONDIE-MAIN raidz2 \69
BLONDIE-4-WD12345/ZFS-BLONDIE-4-WD12345 \
BLONDIE-5-WD12345/ZFS-BLONDIE-5-WD12345 \
BLONDIE-6-WD12345/ZFS-BLONDIE-6-WD12345 \
BLONDIE-7-WD12345/ZFS-BLONDIE-7-WD12345
```

We can add on the mirrors for logging:

```
zpool add BLONDIE-MAIN log mirror \70
BLONDIE-2-WD12345/ZLOG-BLONDIE-2-WD12345 \
BLONDIE-3-WD12345/ZLOG-BLONDIE-3-WD12345
```

Once we get to the point that we have an operating system on disks, and disks ready for use with logs and jails, then we're at the end of this phase of system administration.

Goals for Amnesiac Systems

We chose at this point to do some experimental installs. For clarity, notes on those are not presented here. We did change our goals for the system. We recognized that we wanted:

- Use a LiveCD for an immutable OS
- Encrypt disc partitions instead of the whole disc

Table 1.5: FreeBSD Swap Disk – Estimate of 500 GB HDD

GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	70 GB
Swap	395 GB

Table 1.6: ZFS Cache (Read Cache) Disk – Estimate of 80 GB HDD

GEOM Disk List Mediasize	74 GB
Blank: 15% "Free Space"	11 GB
Swap	63 GB

- Ensure we save keys and metadata to promote disc recovery
- Prepare commands in advance to reduce complexity.

From our testing, we recognizes that a Live CD offered the best answer for an immutable system. It does come at the price of some instability. The "El Torrito" discs have a limited set of services. Adjusting the configuration of the operating system means drafting, burning, and testing a new disc. The " shell" instance on a FreeBSD installation disc is reportedly less stable; but, we did not know why or how it was less stable than a normal operating system. We worked through our disc commands and kept careful notes. Adding the physical serial number of the disc alone increased the complexity of manually typing in each command. We discovered the need to encrypt the discs by partitions instead of the whole disc because there was a visibility problem during the execution of discovering and attaching the discs. There was a poorly documented detail about protecting the disc metadata and keys. The keys are provided during the setup dialog; but, these highly important items will disappear after the setup process closes. This means it is important that an operator recognize the need to intercept and to record these keys. Without them, a later recovery of a disc is impossible. Since one typo would derail our experimental setups, we had to keep a second support computer on hand to record notes and consult references.

As our experimentation concluded, we had learned enough to stand a reasonable chance to setup an amnesiac machine. Its features include:

- An immutable OS
- Encrypted disc partitions
- Swap and logging for ZFS disc features
- Metadata and keys saved to a separate USB drive
- A disc for holding FreeBSD jails (containerization)
- Enough room to expand, to update, and to recover.

Table 1.7: ZFS SLOG Log Mirror (Write Cache) Disk – Initial Estimates

	Estimate for 120 GB SSD
GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

Table 1.8: ZFS Jail Use Hard Disk – Estimate of 500 GB HDD

	Estimate for 500 GB HDD
GEOM Disk List Mediasize	465 GB
Blank: 15% "Free Space"	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

Configure for SWAP Disk, ZFS Cache, ZFS SLOG Mirror, and ZFS RAIDz2

Changed config to keep the RAIDZ2 for main work. I was having trouble with the SSDs; so, I removed them and replaced them with two 80GB Western Digital Velociraptors. I added another WD Velociraptor for cache. Then we added a Western Digital Blue 500GB for a swap disk. That sounds like a lot of swap, but we hope to upgrade to near 196GB of RAM this year. So, a little more than twice that would be 400GB of swap. The next disk size on hand was 500GB.

We had some mis-aligned partitions with RAIDZ2 work for Blondie. We were using the full 500GB on the label as our actual amount of media to work with. Since less is available, we will have to set those up again.

We used `gpart delete` to delete each partition. Then we used `gpart destroy` to delete the partition table for the whole disk. In the end, `gpart` showed us only the CD.

Startup with `bootonly.iso`

For this run, we'll begin by using the boot-only disk. We won't plan to ever boot from these hard drives. We'll just use the shell on the CD. As with the others, we'll mount a USB to store metadata information. We'll partition and encrypt the disks. In the following examples, we provide the actual serial numbers for the hard drives that were used. This example was derived from notes we maintained during experimentation; serial numbers in other instances will vary, of course.

Mount the USB to Save the Metadata Backup File

```
mkdir /tmp/usb371
mount /dev/da0p1 /tmp/usb372
```

Create Swap Disk in Bay 0

```
gpart create -s GPT mfid073  
gpart add -a 5K -s 395G -t freebsd-swap \  
-l ZFS-BLONDIE-0-SWAP-WCC2ERZ56512 mfid074
```

Create ZFS Cache in Bay 1

```
gpart create -s GPT mfid175  
  
gpart add -a 5K -s 63G -t freebsd-zfs \  
-l ZFS-BLONDIE-1-CACHE-WXL1E40C1352 mfid176
```

Create ZFS SLOG in Bays 2 and 3

```
gpart create -s GPT mfid277  
  
gpart add -a 5K -s 63G -t freebsd-zfs \  
-l ZFS-BLONDIE-2-SLOG-WXL1E4067503 mfid278  
  
gpart create -s GPT mfid379  
  
gpart add -a 5K -s 63G -t freebsd-zfs \  
-l ZFS-BLONDIE-3-SLOG-WXC0CA9V8431 mfid380
```

Create ZFS Work Disks for Jails

Bay 4

```
gpart create -s GPT mfid481  
  
gpart add -a 5K -s 50G -t freebsd-swap \  
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid482  
  
gpart add -s 345G -t freebsd-zfs \  
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid483
```

Bay 5

```
gpart create -s GPT mfid584  
  
gpart add -a 5K -s 50G -t freebsd-swap \  
-l ZFS-BLONDIE-5-SWAP-WMC2E3914420 mfid585
```

```
gpart add -s 345G -t freebsd-zfs \  
-l ZFS-BLONDIE-5-FILES-WMC2E3914420 mfid586
```

Bay 6

```
gpart create -s GPT mfid687  
  
gpart add -a 5K -s 50G -t freebsd-swap \  
-l ZFS-BLONDIE-6-SWAP-WCC2ETT39577 mfid688  
  
gpart add -s 345G -t freebsd-zfs \  
-l ZFS-BLONDIE-6-FILES-WCC2ETT39577 mfid689
```

Bay 7

```
gpart create -s GPT mfid790  
  
gpart add -a 5K -s 50G -t freebsd-swap \  
-l ZFS-BLONDIE-7-SWAP-WCAYU2878968 mfid791  
  
gpart add -s 345G -t freebsd-zfs \  
-l ZFS-BLONDIE-7-FILES-WCAYU2878968 mfid792
```

We should note that “Bay 7” is an observation of the physical hardware. The device (“mfid7”) could have whatever number assigned at the end that the machine gave it. If the hardware RAID had a problem with recognizing a device, the number might not correlate to what bay it is in. When powering up, check your geometry.

Encrypt those partitions

During this phase, we’ll encrypt the partitions. That means that we should have some recordkeeping materials on hand. Use a password manager or a notebook to carefully record the password that is about to be typed in. With each one completed, the machine will tell us that it has saved a file of backup metadata. We will want to copy that metadata file to the thumbdrive. If the encryption gets jammed up, or if there is some type of problem, then the metadata file will be used as part of the recovery process. Without a metadata file, recovering an encrypted partition can be difficult or impossible.

If you lose the metadata file, but still have routine access to the geli encryption because you know the password and the machine is functioning alright, then you can export another copy of the metadata. This means that anyone with root access to the machine, who can use geli commands, would also be able to export such a file. Keep this in mind if you need to secure your machine.

When we give the subcommand to “init,” the machine will ask us to give the new password twice. This is part of the password setting process. When we give the subcommand to “attach,” the machine will ask us to give the password as part

of the normal attachment process. Type carefully, and look at the records carefully, because this is the main chance to make sure the password notes are right.

Commands to Encrypt the Partitions

```
geli init -l 256 mfid0p193  
geli attach mfid0p194
```

```
geli init -l 256 mfid1p195  
geli attach mfid1p196
```

```
geli init -l 256 mfid2p197  
geli attach mfid2p198
```

```
geli init -l 256 mfid3p199  
geli attach mfid3p1100
```

```
geli init -l 256 mfid4p1101  
geli attach mfid4p1102
```

```
geli init -l 256 mfid4p2103  
geli attach mfid4p2104
```

```
geli init -l 256 mfid5p1105  
geli attach mfid5p1106
```

```
geli init -l 256 mfid5p2107  
geli attach mfid5p2108
```

```
geli init -l 256 mfid6p1109  
geli attach mfid6p1110
```

```
geli init -l 256 mfid6p2111  
geli attach mfid6p2112
```

```
geli init -l 256 mfid7p1113  
geli attach mfid7p1114
```

```
geli init -l 256 mfid7p2115  
geli attach mfid7p2116
```

To Preserve the Metadata Backup Files

Let us reiterate: preserving these backups at the time of disk creation is essential for later recoveries. If these files are not saved, then a powerdown of the system will cause them to be lost. They're automatically stored in an ephemeral directory. Conserve the ability to recover a disk later by recording these metadata backup files to a safe location.

Each metadata backup file was probably created with a name that does not look like the drive's label. Instead, the machine assigns a backup file name based on geometry. Since we can have a hardware configuration change that could mess up that pattern, we will want to save our backup files with names that look more like the disk labels. That way, if the drives move, then we can better discover which metadata file goes with it.

```
cp /var/backups/mfid0p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-0-SWAP-WCC2ERZ56512.eli117
```

```
cp /var/backups/mfid1p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-1-CACHE-WXL1E40C1352.eli118
```

```
cp /var/backups/mfid2p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-2-SLOG-WXL1E4067503.eli119
```

```
cp /var/backups/mfid3p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-3-SLOG-WXC0CA9V8431.eli120
```

```
cp /var/backups/mfid4p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-4-SWAP-WCC2EE134983.eli121
```

```
cp /var/backups/mfid4p2.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-4-FILES-WCC2EE134983.eli122
```

```
cp /var/backups/mfid5p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-5-SWAP-WMC2E3914420.eli123
```

```
cp /var/backups/mfid5p2.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-5-FILES-WMC2E3914420.eli124
```

```
cp /var/backups/mfid6p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-6-SWAP-WCC2ETT39577.eli125
```

```
cp /var/backups/mfid6p2.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-6-FILES-WCC2ETT39577.eli126
```

```
cp /var/backups/mfid7p1.eli /tmp/usb3/backups  
_27SEP2020/ZFS-BLONDIE-7-SWAP-WCAYU2878968.eli127
```

```
cp /var/backups/mfid7p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-7-FILES-WCAYU2878968.eli128
```

File Name Check: Editing Labels

We got through all that, and found that we had named a file incorrectly. During experimentation, we had a typo; BAY 4 should be `WCC2EE134983` but that's not what we saw when we checked our work. Not the end of the world. We can move the `.eli` metadata file, and we can use `gpart`¹²⁹ `modify` to change the label.

```
gpart modify -i 1 \
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid4130
```

```
gpart modify -i 2 \
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid4131
```

`gpart show`¹³² and `geli status`¹³³ should show our drives as we expect them to be.

At this point, all of the drives should be partitioned and decrypted and attached. We can build a `zpool` for our ZFS file system. We can attach that swap drive for the benefit of our operating system right now.

To Use the Swap Disk

```
swapon mfid0p1.eli134 swapinfo -k135
```

64G turned out to be the maximum capacity. We will need to fix up the swap better. We can fit 6 x 64G (384G) of swap partitions in our planned space. So, let's modify and add.

```
gpart resize -i 1 -s 64G mfid0136
```

We would need to add, to `init` and to attach five more swap partitions. We also have to restore `mfid0` because there's a problem with reading the metadata.

```
gpart recover mfid0137
gpart status mfid0138
geli attach mfid0p1139
```

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-B-WCC2ERZ56512 mfid0140
```

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-C-WCC2ERZ56512 mfid0141
```

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-D-WCC2ERZ56512 mfid0142
```

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-E-WCC2ERZ56512 mfid0143
```

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-F-WCC2ERZ56512 mfid0144
```

Then we need to establish encryption for all of those.

```
geli init -l 256 mfid0p2145
geli attach mfid0p2146
geli init -l 256 mfid0p3147
geli attach mfid0p3148
geli init -l 256 mfid0p4149
geli attach mfid0p4150
geli init -l 256 mfid0p5151
geli attach mfid0p5152
geli init -l 256 mfid0p6153
geli attach mfid0p6154
```

Using `swapinfo`¹⁵⁵, `swapon`¹⁵⁶ and `swapoff`¹⁵⁷, we were able to remove some of the other swap partitions and add on these. Did see a kernel limit on swap. Total configured swap exceeded `kern.maxswzone` (98087976 pages). We could tune the kernel, or we could reduce swap. For now, we reduce swap from 384GB to 324GB.

Similarly, we could add on the other swap partitions, although in our current use, we probably won't need them. They would serve us better in cases where we were doing more intensive work, or working on another machine. We can mount them now, anyway, to see that it can be done.

To add on all the other swap partitions:

```
swapon /dev/mfid4p1.eli158
swapon /dev/mfid5p1.eli159
swapon /dev/mfid6p1.eli160
swapon /dev/mfid7p1.eli161
swapinfo -h162
```

Create the zpool for BLONDIE

We'll now create a mountpoint on the tmpfs, create the zpool, and then add in the cache and SLOG Mirrored disks.

Create a mountpoint

Create a mountpoint so that we will be able to mount the zpool without errors as we create it.

```
mkdir /tmp/BLONDIE163
```

1.4 Analysis

1.4.1 Quality

1.4.2 Tests

1.5 Discussion

1.6 Chapter References

1.6.1 Books

For our ZFS planning, we read: Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.

1.6.2 MAN Pages

Online MAN pages were available at: man.freebsd.org with an interface that allows for keyword based searches.

“bsdinstall.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“camcontrol.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“cd.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“cp.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“geli(8).” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

“gpart(8).” Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-R

RELEASE&arch=default&format=html. Accessed 10 June 2026.

“ls.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“mkdir.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“mount.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“mt.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mt&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“newfs.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“rm.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“ssh-Keygen.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“su.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=su&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“swapon.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“umount.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“zfs.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

“zpool.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

1.6.3 Websites

When planning and exploring our use of swap, we consulted: <https://www.cyberciti.biz/faq/create-a-freebsd-swap-file/>.

Endnotes

- ¹Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 3, no. 2, 30 June 2022, pp. 176-184, <https://doi.org/10.63282/3050-9262.ijaidsm1-v3i2p119>. Accessed 21 June 2026.
- ²Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 3, no. 2, 30 June 2022, pp. 176-184, <https://doi.org/10.63282/3050-9262.ijaidsm1-v3i2p119>. Accessed 21 June 2026.
- ³Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 3, no. 2, 30 June 2022, pp. 176-184, <https://doi.org/10.63282/3050-9262.ijaidsm1-v3i2p119>. Accessed 21 June 2026.
- ⁴Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 3, no. 2, 30 June 2022, pp. 176-184, <https://doi.org/10.63282/3050-9262.ijaidsm1-v3i2p119>. Accessed 21 June 2026.
- ⁵Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 3, no. 2, 30 June 2022, pp. 176-184, <https://doi.org/10.63282/3050-9262.ijaidsm1-v3i2p119>. Accessed 21 June 2026.
- ⁶Chandramouli, Ramaswamy. Guidelines for Media Sanitization. 2025, nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-88r2.pdf, <https://doi.org/10.6028/nist.sp.800-88r2>.
- ⁷"Blondie | the Official Website of Blondie, Featuring Tour Dates, Presale Ticketing, News, the Official Store and More." *Blondie.net*, 2026, blondie.net/. Accessed 10 June 2026.
- ⁸"LTO Ultrium 3: Product Views | Fujifilm [United States]." *Fujifilm.com*, 2026, www.fujifilm.com/us/en/business/data-storage/data-storage-media/lto-ultrium-3/product-views. Accessed 10 June 2026.
- ⁹"WD VelociRaptor © SATA Hard Drives" <https://theretroweb.com/storage/documentation/wd6000-velociraptor-68407cb5c0c96653507203.pdf>
- ¹⁰Intel® Xeon® Processor 5500 Series an Intelligent Approach to IT Challenges. www.intel.com/content/dam/doc/product-brief/xeon-5500-server-brief.pdf.
- ¹¹Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ¹²This pattern of coordinating host computer, bay position, and drive serial number was recommended by Lucas and Jude. Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ¹³Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ¹⁴zfs set shown in: "Zfs." *Freebsd.org*, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹⁵Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ¹⁶zfs set shown in: "Zfs." *Freebsd.org*, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹⁷"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." *Docs.oracle.com*, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.
- ¹⁸Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ¹⁹"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." *Docs.oracle.com*, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.
- ²⁰Lucas, Michael W, and Allan Jude. *FreeBSD Mastery: ZFS*. Tilted Windmill Press, 21 May 2015.
- ²¹"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." *Docs.oracle.com*, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/

[index.html](#).

²²Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

²³“Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide).” [Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html](#).

²⁴Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

²⁵zfs set shown in: “Zfs.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

²⁶“Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide).” [Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html](#).

²⁷Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

²⁸zfs set shown in: “Zfs.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

²⁹“Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide).” [Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html](#).

³⁰Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

³¹zfs set shown in: “Zfs.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

³²“Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide).” [Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html](#).

³³Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

³⁴zfs set shown in: “Zfs.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

³⁵“Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide).” [Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html](#).

³⁶Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

³⁷zfs set shown in: “Zfs.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

³⁸Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

³⁹Lucas and Jude provide a detailed breakdown of different RAID and RAID-Z combinations with differing numbers of drives. Their analysis and provided charts were our chief reason for settling in on RAIDZ2. Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

⁴⁰“bsdinstall.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

⁴¹*Ibid.*

⁴²“bsdinstall.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html](#)

⁴³“bsdinstall.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html](#)

⁴⁴“ssh-Keygen.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

⁴⁵“camcontrol.” [Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html](#). Accessed 10 June 2026.

⁴⁶“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026

⁴⁷Ibid.

⁴⁸“newsf.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁴⁹“Mkdir.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁰“Mount.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵¹“cd.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵²“umount.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵³“rm.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁴“Mkdir.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁵“Mount.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁶“ls.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁷“Mkdir.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁸“ssh-Keygen.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁵⁹“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁶⁰“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁶¹Ibid.

⁶²Ibid.

⁶³Ibid.

⁶⁴Ibid.

⁶⁵Ibid.

⁶⁶Ibid.

⁶⁷Ibid.

⁶⁸“zpool.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁶⁹Ibid.

⁷⁰Ibid.

⁷¹“Mkdir.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html

=html. Accessed 10 June 2026.

⁷²“mount.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁷³“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

⁷⁴Ibid.

⁷⁵Ibid.

⁷⁶Ibid.

⁷⁷Ibid.

⁷⁸Ibid.

⁷⁹Ibid.

⁸⁰Ibid.

⁸¹Ibid.

⁸²Ibid.

⁸³Ibid.

⁸⁴Ibid.

⁸⁵Ibid.

⁸⁶Ibid.

⁸⁷Ibid.

⁸⁸Ibid.

⁸⁹Ibid.

⁹⁰Ibid.

⁹¹Ibid.

⁹²Ibid.

⁹³“Geli(8).” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

⁹⁴Ibid.

⁹⁵Ibid.

⁹⁶Ibid.

⁹⁷Ibid.

⁹⁸Ibid.

⁹⁹Ibid.

¹⁰⁰Ibid.

¹⁰¹Ibid.

¹⁰²Ibid.

¹⁰³Ibid.

¹⁰⁴Ibid.

¹⁰⁵Ibid.

¹⁰⁶Ibid.

¹⁰⁷Ibid.

¹⁰⁸Ibid.

¹⁰⁹Ibid.

¹¹⁰Ibid.

¹¹¹Ibid.

¹¹²Ibid.

¹¹³Ibid.

¹¹⁴Ibid.

¹¹⁵Ibid.

¹¹⁶Ibid.

¹¹⁷“cp.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

¹¹⁸Ibid.

¹¹⁹Ibid.

¹²⁰Ibid.

- ¹²¹Ibid.
- ¹²²Ibid.
- ¹²³Ibid.
- ¹²⁴Ibid.
- ¹²⁵Ibid.
- ¹²⁶Ibid.
- ¹²⁷Ibid.
- ¹²⁸Ibid.
- ¹²⁹“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&propos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹³⁰Ibid.
- ¹³¹Ibid.
- ¹³²Ibid.
- ¹³³“geli(8).” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
- ¹³⁴“swapon.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&propos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹³⁵Ibid.
- ¹³⁶“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&propos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹³⁷“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&propos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹³⁸“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&propos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹³⁹“geli(8).” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
- ¹⁴⁰“gpart(8).” FreeBSD.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&propos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹⁴¹Ibid.
- ¹⁴²Ibid.
- ¹⁴³Ibid.
- ¹⁴⁴Ibid.
- ¹⁴⁵“geli(8).” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
- ¹⁴⁶Ibid.
- ¹⁴⁷Ibid.
- ¹⁴⁸Ibid.
- ¹⁴⁹Ibid.
- ¹⁵⁰Ibid.
- ¹⁵¹Ibid.
- ¹⁵²Ibid.
- ¹⁵³Ibid.
- ¹⁵⁴Ibid.
- ¹⁵⁵“swapon.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&propos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
- ¹⁵⁶Ibid.
- ¹⁵⁷Ibid.
- ¹⁵⁸“swapon.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&propos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

¹⁵⁹Ibid.

¹⁶⁰Ibid.

¹⁶¹Ibid.

¹⁶²Ibid.

¹⁶³“mkdir.” FreeBSD.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

2 FreeBSD Tape Operations

We explored the planning and use of LTO tape as a resource for storing backups. We describe common commands, patterns of MT and TAR commands. We recommend some techniques for testing, rehearsing, and using tape archives.

2.1 Introduction

We wanted to see if it would be possible for us to make use of an LTO tape machine that was on one of our salvaged computers. We'd often heard of the performance of using tape for backups. We'd seen some hints about the bravery needed for working with tape in modifying old mainFRAME programs. We rarely saw any strong tutorials or comprehensive directions for tape operations.

The average person has almost never used tapes as a part of personal computing these days. Businesses which do use tapes often do so as part of larger, commercial data center operations. For many of us, the closest we've come to tape operations was with using cassette tapes for storage on personal machines in the 1980s. LTO tapes are often praised but infrequently used. We wanted to break through this mystery.

2.2 Methodology

Our plan is to write a file to tape, eject the tape, and be able to read the file back from the tape again. Once there, we will want to do a simple sequence of file writes and reads without damaging the files on tape.

2.3 Results

We used a Dell T610 tower computer with an LT03 single deck tape machine on board. We have some LT03 tapes that can be used with it. Our tape drivers and common commands are built into FreeBSD.

Warning: We want to use the nsa driver. There is an sa driver, but it has some limitations. When we tried using sa, we were not able to properly advance down the tape through multiple file markers. Use nsa for the the driver.

2.3.1 Code Notes

1. Place files in a directory that you control. Isolating the files to be copied to tape is safer than copying them directly from where they are.

2. `mt -f /dev/nsa0 load`

This will load the tape onto the machine.

3. `mt -f /dev/nas0 rewind`

This will spool the tape to the beginning.

4. `mt -f /dev/nsa0 status`

To check to see where we are on the tape. Assuming this is the first use of the tape, we could start writing immediately.

5. If there are files on the tape, we need to either scan ahead and log the files, or we need to consult our notes to find where on the tape we should go. We could easily overwrite a file and damage the sequence. Use the write persistent notch to protect the tape when possible.

6. `mt -f /dev/nsa0 com or eod`

To go to the end of the records.

7. `tar cvf /dev/nsa0 [FILE_TO_ARCHIVE]`

Once at the proper spot on the tape, give the command to archive the files.

8. `mt -f /dev/nsa0 weof1`

Write an end-of-file marker.

9. `tar -t`

To see the archive on the tape.

`tar xf /dev/nsa0`

To get the archive from the tape.

10. Write notes. Log and record what was put on the tape.

11. For the next record, just rewind and advance through the markers to get to the next blank space for writing a record.

Sample Files: test.txt

The quick brown fox jumped over the lazy dogs.

Sample Files: test2.txt

The other dog is still lazy, too.

Sample Files: test3.txt

The fox is just really fast.

Sample Files: test4.txt

Yet some more records.

Sample Files: test5.txt

And another record.

Sample Script: scriptedDemo.sh

```
#!/usr/bin/env sh
clear
echo "scriptedDemo"
echo "Press ENTER to continue."
```

```
read INPUT
## Check tape
echo "Check if tape is staged in machine."
echo "Press ENTER to continue."
read INPUT
clear
## Load the tape into the machine
echo "Here we go."
echo
echo "Loading tape into machine."
echo "mt -f /dev/nsa0 load"
mt -f /dev/nsa0 load
## Rewind tape
echo "Rewinding tape"
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "Press ENTER to continue."
read INPUT
clear
echo "Reading status from tape."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 0 on the tape."
echo "We should be at the beginning."
echo "So, no mt commands this time."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 1 on tape."
echo "Rewind to the beginning."
mt -f /dev/nsa0 rewind
echo "Go forward space count 1 file."
echo "mt -f /dev/nsa0 fsf 1"
mt -f /dev/nsa0 fsf 1
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
```

```
clear
echo "Looking for file 2 on tape."
echo "Back up to 0 and go forward space count 2 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 2"
mt -f /dev/nsa0 fsf 2
echo "Show where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Look in that spot with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for third archive on tape."
echo "Back up to 0 and go forward space count 3 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 3"
mt -f /dev/nsa0 fsf 3
echo "See where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "See what is here with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Take the tape offline."
echo "Rewind tape."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 offline"
mt -f /dev/nsa0 offline
exit 0
```

2.4 Analysis

2.4.1 Quality

1. We found there were significant weaknesses in tape sequencing. Unlike random access memory, overwriting can damage end of file markers which are only navigational delimiters readily usable with the tape. Simple practice showed it was very easy to accidentally overwrite a tape marker. All subsequent files on the tape might be inaccessible as a result.

2. Logging. Since there may be many files on these long tapes, logging support and file navigation are essential. The usual lapses in notation can make frequent use of the tapes more hazardous with each use.

3. Security. Unlike geli-encrypted hard drives, the main file protection scheme is through whatever encryption we apply to the files themselves. Tar has some compression algorithms, but encrypting files is another step altogether. Since the files may be in storage for a long time, having the backups encrypted on tape can be a critical step. This encryption would need to be applied before IV.B.7. tar or archiving the files to tape.

2.4.2 Tests

Writing to tape should be rehearsed. Checking status and reading files to confirm our location on the tape is a good idea.

Any write operation should be couched on a good, rehearsed read script. Isolate the files to be written to their own directory. Add an encryption step if necessary. Rehearse the tar tape archive commands. Compartmentalize the steps of the tape operations plan to reduce catastrophic failures and a loss of data.

2.5 Discussion

Tape operations have been long recommended as the preferred form of backup. Off-site backup offers strong advantages to an on-premises system operator. We can't count the number of times we've seen backups recommended; yet we find very few practical tutorials or directions.

Tape backups and hard drive backups might be easily and securely routed to an off site location through the US mail. They could be mailed to yourself at a personal mailbox, remote office, P.O. Box, or to a trusted person. Notice that any person or entity entrusted to possess the data may also be at risk or some sort of liability.

2.6 References

A. Books – none. 2023 versions of the FreeBSD handbook don't cover tape operations.

B. man pages – `mt`, `tar` man pages.

C. URLs – FreeBSD man pages for above. man.freebsd.org/cgi/man.cgi?query=mt&tar

Text with¹ inline references. More text²

Endnotes

¹First note.

²Second note.

Glossary

amnesiac The Oxford English Dictionary (OED) described amnesiac as (1913-), "One who is afflicted with amnesia." ¹ In this context, an amnesiac system is one that will forget recent changes; usually, the system has an immutable operating system as a defense against attacks.

BSD Berkeley Software Distribution, a Unix-derived operating system and its variants (FreeBSD, OpenBSD, NetBSD).

cache The OED described it as, "3. 1968- Computing. A small high-speed memory in some computers into which are placed the most frequently accessed contents of the slower main memory or secondary storage. Also cache memory." ²

CCSP Certified Cloud Security Professional.

CISSP Certified Information Systems Security Professional.

coding The OED described coding as (1946-), "3. Computing. The conversion of information into code usable by a computer or other machine; the writing or editing of code for a computer program, application, etc. Also: computer code." ³

configuration The OED described configuration as (1962), "8. Computing. The way the constituent parts of a computer system are chosen or interconnected in order to suit it for a particular task or use; the units or devices required for this." ⁴

CSSLP Certified Secure Software Lifecycle Professional.

dialog OED(1) says, "4.b. 1984- Chiefly in form dialog. = dialogue box n." ⁵ OED(2) says, "Computing. 1984- A window in a graphical user interface which asks the user for input in response to some given information, such as a choice of command in answer to a question." ⁶

encrypt The OED described it as, "transitive. To convert (data, a message, etc.) into cipher or code, esp. in order to prevent unauthorized access; to conceal in something by this means." ⁷

GPU Graphics Processing Unit.

immutable system The OED described immutable as (1621), "1.b. technical. Not subject to variation in different cases; invariable: used e.g. of markings which are the same in all the individuals of a species." ⁸ An immutable system is one that cannot be changed. In these computing systems, the intent of using an immutable system is to allow an operator to restore a system to its original state by power cycling it.

jail A FreeBSD OS-level virtualisation mechanism that partitions the operating system into isolated environments.

memory stick The OED described it as, "(Originally) a type of memory card for use in digital cameras and other devices; (now chiefly) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism. A proprietary name."⁹

metadata The OED described it as, "Data whose purpose is to describe and give information about other data."¹⁰

mirror OED described it as (1993-), "4. transitive. Computing. To write (data) on to two separate devices (esp. two hard disks) simultaneously, to protect against the possible failure of one; to create (a duplicate disk) in this manner. Also: to copy (a website) on to a different server (cf. mirror site n.)."¹¹

motherboard The OED described it as (1965-), "Electronics. A printed circuit board on which the principal components of a microcomputer or other electronic device are mounted, and to which other boards (daughterboards) may be connected."¹²

NIC Network Interface Card.

NIST National Institute of Standards and Technology.

OED Oxford English Dictionary

operating system The OED described it as (1961-), "Computing. The low-level (low-level adj. 4b) software that supports a computer's basic functions, such as scheduling tasks, controlling peripherals, and allocating storage."¹³

pipe The OED had a couple relevant descriptions for it. "III.26.b. 1979- A physical or notional data channel or route."¹⁴ Also, "III.26.c. 1982- A temporary connection between two processes or commands, so that the output from one command becomes the input for the next."¹⁵

Poudriere Named after the French word for "powderkeg," Poudriere is a package building program that will custom-compile programs from ports and configuration files. A FreeBSD operator can then use their own tailored programs built to specifications while obtaining updates from the ports tree.

RAID The OED described it as (1988-), "Computing. Redundant array of inexpensive disks (or drives), a data storage system which combines multiple hard drives to form a system functioning as single drive."¹⁶

RAM Random Access Memory.

SAS Serial Attached SCSI.

SATA Serial AT Attachment.

shell The OED described it as (1974-), "b. In some operating systems (originally Multics and Unix): a program that translates commands keyed by the user into commands that the operating system can act on, thereby providing a high-level interface with the user. Also (more fully shell window): a window in which such commands can be invoked."¹⁷

SSD Solid State Drive.

SSH Secure Shell.

thumbdrive OED described it as, "a. A mechanism that is operated by the thumb; b. Computing (a proprietary name for) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism; cf. memory stick n." ¹⁸

VNET Virtual Network.

Endnotes

- ¹ “Amnesiac, N.” Oxford English Dictionary, Oxford UP, December 2024, <https://doi.org/10.1093/OED/9207200444>.
- ² “Cache, N. (2), Sense 3.” Oxford English Dictionary, Oxford UP, September 2025, <https://doi.org/10.1093/OED/1117661132>.
- ³ “Coding, N., Sense 3.” Oxford English Dictionary, Oxford UP, June 2025, <https://doi.org/10.1093/OED/4701717734>.
- ⁴ “Configuration, N., Sense 8.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/1065280211>.
- ⁵ “Dialogue | Dialog, N., Sense 4.b.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/4999880636>.
- ⁶ “Dialogue Box | Dialog Box, N.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/3367209004>.
- ⁷ “Encrypt, V.” Oxford English Dictionary, Oxford UP, December 2025, <https://doi.org/10.1093/OED/1124748007>.
- ⁸ “Immutable, Adj., Sense 1.b.” Oxford English Dictionary, Oxford UP, December 2025, <https://doi.org/10.1093/OED/1176087402>.
- ⁹ “Memory Stick, N.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/1531282927>.
- ¹⁰ “Metadata, N.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/7968104326>.
- ¹¹ “Mirror, V., Sense 4.” Oxford English Dictionary, Oxford UP, December 2025, <https://doi.org/10.1093/OED/8366938443>.
- ¹² “Motherboard, N.” Oxford English Dictionary, Oxford UP, July 2023, <https://doi.org/10.1093/OED/1154882806>.
- ¹³ “Operating System, N.” Oxford English Dictionary, Oxford UP, July 2023, <https://doi.org/10.1093/OED/4828064025>.
- ¹⁴ “Pipe, N. (1), Sense III.26.b.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/5943533372>.
- ¹⁵ “Pipe, N. (1), Sense III.26.c.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/8044200378>.
- ¹⁶ “RAID, N.” Oxford English Dictionary, Oxford UP, June 2026, <https://doi.org/10.1093/OED/4981410541>.
- ¹⁷ “Shell, N., Additional sense.” Oxford English Dictionary, Oxford UP, March 2026, <https://doi.org/10.1093/OED/4563192747>.
- ¹⁸ “Thumb Drive, N.” Oxford English Dictionary, Oxford UP, March 2026, <https://doi.org/10.1093/OED/9979951645>.

Index

2FA, 8

amensiac, 1, 5

bsdinstall, 7, 8

cache, 13, 14, 20

camcontrol

 devlist, 10

cd, 10

coding, 1, 5

cp, 18, 19

dd, 2

dialog, 11, 13

encrypt, 12–14, 16, 17, 20, 33

FreeBSD, 1, 2, 4, 13, 29

FreeBSD Handbook, 9, 11, 34

FreeBSD Mastery: ZFS, 5

geli, 3, 16

 attach, 17, 19, 20

 init, 17, 20

 status, 19

gpart, 5, 7, 11, 14

 add, 10–12, 15, 16, 19, 20

 create, 10, 11, 15, 16

 modify, 19

 recover, 19

 resize, 19

 show, 19

 status, 19

jails, 8

live CD, 1

ls, 10

LTO III tape, 3

mkdir, 10, 11, 14

mount, 10, 14

newfs, 10

NIST 800-88

 Purge, 2

operating system, 1, 5, 7, 8, 12, 13, 19

RAIDZ-2, 3–5, 11, 12

rm, 10

ssh-keygen, 9, 11

su, 9

sudo, 9

swapinfo, 19, 20

swapoff, 20

swapon, 11, 19, 20

TOTP, 8

umount, 10

USB

 drive, 8

zpool, 19–21

 add, 12

 comment, 6

 create, 12